

## MS SQL Joins Part 2

By Don Schlichting

### Introduction

In Part 1 of this series, the purposes and reasons for using Joins were explored. The advantages and disadvantages of Joins were also discussed. The conclusion was that SQL Server transactions will perform well when the database is designed with multiple related tables, as opposed to a single, large, multi-field table. If this design is followed, then Joins will be needed to extract data for reports and viewing. The Joins will allow data from multiple tables to be logically connected to each other and return only one record set, regardless of the number of individual tables used in a statement.

Use the following statement to create the test database used in this article.

```
USE MASTER;
GO
CREATE DATABASE JoinTest;
GO
USE JoinTest;
GO
CREATE TABLE Customers
(CustID int, CustomerName varchar(50),Addr varchar(50),City varchar(50) );
CREATE TABLE Products
(ProdID int, ProductName varchar(50),Descr varchar(50),Price money );
CREATE TABLE Sales
(SaleDate datetime, ProdID int, CustID int, Quantity int );
GO
INSERT INTO Customers VALUES (1, 'Mr Smith', '123 Elm St', 'Milwaukee');
INSERT INTO Customers VALUES (2, 'Mr Jones', '456 South St', 'Chicago');
INSERT INTO Customers VALUES (3, 'Mr Brown', '789 Oak St', 'St Louis');
INSERT INTO Products VALUES (4, 'Green Widgit', 'Lagre Green', 100);
INSERT INTO Products VALUES (5, 'Blue Widgit', 'XLagre Blue', 130);
INSERT INTO SALES VALUES ('01/01/07', 4, 1, 1);
INSERT INTO SALES VALUES ('02/02/07', 4, 1, 2);
INSERT INTO SALES VALUES ('02/02/07', 5, 1, 3);
INSERT INTO SALES VALUES ('01/01/07', 5, 2, 1);
```

The statement creates the three tables and sample data shown below. For the Customers table, we will be joining on the Int of the Customer ID. An Int is not required for a Join though; the Join could also have been done on the Customer Name instead. Also note that a Primary or Foreign key is not required for a Join. However, both Ints and keys will increase performance.

| Table - dbo.Customers     |                    |               |              |           |
|---------------------------|--------------------|---------------|--------------|-----------|
|                           | CustID             | CustomerName  | Addr         | City      |
| ▶                         | 1                  | Mr Smith      | 123 Elm St   | Milwaukee |
|                           | 2                  | Mr Jones      | 456 South St | Chicago   |
|                           | 3                  | Mr Brown      | 789 Oak St   | St Louis  |
| 1 of 3 <span>▶▶▶▶▶</span> |                    |               |              |           |
| Table - dbo.Products      |                    |               |              |           |
|                           | ProdID             | ProductName   | Descr        | Price     |
| ▶                         | 4                  | Green Widgeit | Lagre Green  | 100.0000  |
|                           | 5                  | Blue Widgeit  | XLagre Blue  | 130.0000  |
| 1 of 2 <span>▶▶▶▶▶</span> |                    |               |              |           |
| Table - dbo.Sales         |                    |               |              |           |
|                           | SaleDate           | ProdID        | CustID       | Quantity  |
|                           | 1/1/2007 12:00:... | 4             | 1            | 1         |
|                           | 2/2/2007 12:00:... | 4             | 1            | 2         |
|                           | 2/2/2007 12:00:... | 5             | 1            | 3         |
|                           | 1/1/2007 12:00:... | 5             | 2            | 1         |

The example from part one of this series used an Inner Join to display all the customers who had a purchase recorded in the Sales table. In this next example, all the customers, including those that have not made a purchase will be displayed.

## LEFT OUTER JOIN

Unlike Inner Joins, where each table being joined must have at least one matching row, Outer Joins return all the rows from one table, and any matching rows from another. In this first example, we are looking for each Customer Name in the Customers database, regardless if they have purchased or not. If they have purchased, show the purchase date.

```
SELECT Customers.CustomerName, Sales.SaleDate
FROM Customers
LEFT OUTER JOIN Sales ON Sales.CustID = Customers.CustID
```

The above TSQL will produce the desired result set as show below.

|   | CustomerName | SaleDate                |
|---|--------------|-------------------------|
| 1 | Mr Smith     | 2007-01-01 00:00:00.000 |
| 2 | Mr Smith     | 2007-02-02 00:00:00.000 |
| 3 | Mr Smith     | 2007-02-02 00:00:00.000 |
| 4 | Mr Jones     | 2007-01-01 00:00:00.000 |
| 5 | Mr Brown     | NULL                    |

The statement achieved the desired results. All the customer names have been included, including Mr Brown who has not purchased. Because there was no purchase for him, the join returned NULL as the sales date. The TSQL statement starts like a normal Select, naming the columns to be returned. Notice we have fully qualified the name of each column, indicating what table it belongs to, such as Customers.CustomerName. This isn't really necessary in this example. We could have stated: "SELECT CustomerName, SaleDate", and the statement would have executed correctly. Fully qualifying becomes important when you select a column that exists in both tables, such as CustID. In that type of case, indicating which table the column belongs to is required. For example, the following Select produces an error:

```
SELECT CustID
FROM Customers
LEFT OUTER JOIN Sales ON Sales.CustID = Customers.CustID
```

Msg 209, Level 16, State 1, Line 1  
Ambiguous column name 'CustID'.

In order to remove the error, we must specify from which table CustID should be returned, Customers or Sales. Typically, a good coding practice is to always fully qualify each column. Table aliases can also be used for fully qualifying. The next article in this series will give examples of aliasing.

The next line in the working TSQL statement joins the Customers and the Sales tables:

```
FROM Customers LEFT OUTER JOIN Sales ON Sales.CustID = Customers.CustID
```

With a LEFT OUTER JOIN, all the rows from the table on the left of the clause will be returned. In addition, any matching rows on the right will also be returned. The order of the tables after the key word "ON" doesn't matter.

## **RIGHT OUTER JOIN**

A Right Join performs the same function as a Left. The only difference is that all rows from the right of the clause are returned, rather than the left. The previous statement could be rewritten using a Right Join as follows:

```
SELECT Customers.CustomerName, Sales.SaleDate
FROM Sales
RIGHT OUTER JOIN Customers ON Sales.CustID = Customers.CustID
```

This statement produces the exact same results as the Left Outer Join example.

## **FULL OUTER JOIN**

This last outer join returns all the data from both tables, matches and non-matches alike. To demonstrate this, insert an additional row of data into sales, a row without a customer.

```
INSERT INTO SALES VALUES ('01/01/07', 0, 0, 1);
```

Next Select using a Full Outer Join:

```
SELECT Customers.CustomerName, Sales.SaleDate
FROM Sales
FULL OUTER JOIN Customers ON Sales.CustID = Customers.CustID
```

|   | CustomerName | SaleDate                |
|---|--------------|-------------------------|
| 1 | Mr Smith     | 2007-01-01 00:00:00.000 |
| 2 | Mr Smith     | 2007-02-02 00:00:00.000 |
| 3 | Mr Smith     | 2007-02-02 00:00:00.000 |
| 4 | Mr Jones     | 2007-01-01 00:00:00.000 |
| 5 | NULL         | 2007-01-01 00:00:00.000 |
| 6 | Mr Brown     | NULL                    |

The Full Outer Join returned the records where there was a match in both tables (rows 1 – 4), the newly inserted record where there was no customer (row 5), and the record for Mr Brown who has never purchased (row 6).

## Conclusion

If you have a high performance database doing transactions, then your table schema will include several tables that logically relate to each other. In this type of situation, Joins are required to return datasets that combine multi-table data into one logical rowset. Joins are always between two tables. More than one Join can be used to link three or more tables together. The next article in this series will have examples of this. An Inner Join will be used to return data from two tables where some piece of data is an exact match in both tables. An Outer Join is used to return all the data from one table, and exact matches from the other. In the next article, we will work with Join Hints, Cross Joins, and using Joins in Update and Delete statements.